Attention Is All You Need

The Transformer Architecture that Revolutionized NLP

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit,
Llion Jones, Aidan N. Gomez, Łukasz Kaiser, Illia Polosukhin

Google Brain & Google Research

Presented by-
Sajjan Singh

Presentation Outline

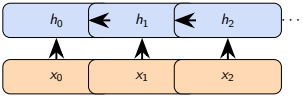
1. Introduction
2. The Transformer Architecture
3. Attention Mechanism
4. Position-wise Feed-Forward Networks
5. Embeddings and Positional Encoding
6. Why Self-Attention?
7. Training and Results
8. Conclusion and Impact

The Pre-Transformer Era

Dominant Sequence Models (Before 2017):

- **Recurrent Neural Networks (RNNs)**
  - Process sequences step-by-step
  - Hidden state: $h_t = f(h_{t-1}, x_t)$

RNN Sequential Processing

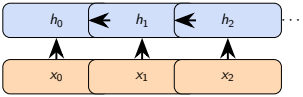


The Pre-Transformer Era

Dominant Sequence Models (Before 2017):

- **Recurrent Neural Networks (RNNs)**
  - Process sequences step-by-step
  - Hidden state: $h_t = f(h_{t-1}, x_t)$
- **Long Short-Term Memory (LSTM)**
  - Addresses vanishing gradient problem
  - Introduced gating mechanisms

RNN Sequential Processing

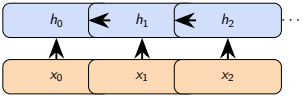


The Pre-Transformer Era

Dominant Sequence Models (Before 2017):

- **Recurrent Neural Networks (RNNs)**
  - Process sequences step-by-step
  - Hidden state: $h_t = f(h_{t-1}, x_t)$
- **Long Short-Term Memory (LSTM)**
  - Addresses vanishing gradient problem
  - Introduced gating mechanisms
- **Gated Recurrent Units (GRU)**
  - Simplified version of LSTM
  - Fewer parameters

RNN Sequential Processing

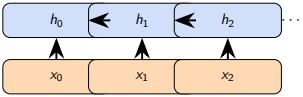


The Pre-Transformer Era

Dominant Sequence Models (Before 2017):

- **Recurrent Neural Networks (RNNs)**
  - Process sequences step-by-step
  - Hidden state: $h_t = f(h_{t-1}, x_t)$
- **Long Short-Term Memory (LSTM)**
  - Addresses vanishing gradient problem
  - Introduced gating mechanisms
- **Gated Recurrent Units (GRU)**
  - Simplified version of LSTM
  - Fewer parameters
- **Convolutional Seq2Seq**
  - Parallel computation possible
  - Limited receptive field

RNN Sequential Processing



The Pre-Transformer Era

Dominant Sequence Models (Before 2017):

- **Recurrent Neural Networks (RNNs)**
  - Process sequences step-by-step
  - Hidden state: $h_t = f(h_{t-1}, x_t)$
- **Long Short-Term Memory (LSTM)**
  - Addresses vanishing gradient problem
  - Introduced gating mechanisms
- **Gated Recurrent Units (GRU)**
  - Simplified version of LSTM
  - Fewer parameters
- **Convolutional Seq2Seq**
  - Parallel computation possible
  - Limited receptive field

Core Problem

Sequential computation prevents parallelization!

- Long sequences $\rightarrow$ slow training
- Memory constraints limit batching
- Difficulty learning long-range dependencies

The Fundamental Problem with RNNs

Sequential Nature:

$$h_t = f(h_{t-1}, x_t)$$

The Fundamental Problem with RNNs

Sequential Nature:

$$h_t = f(h_{t-1}, x_t)$$

Implications:

- Cannot parallelize within training examples

The Fundamental Problem with RNNs

Sequential Nature:

$$h_t = f(h_{t-1}, x_t)$$

Implications:

- Cannot parallelize within training examples
- Memory constraints limit batching across examples

The Fundamental Problem with RNNs

Sequential Nature:

$$h_t = f(h_{t-1}, x_t)$$

Implications:

- Cannot parallelize within training examples
- Memory constraints limit batching across examples
- Long sequences $\rightarrow$ long computation paths

The Fundamental Problem with RNNs

Sequential Nature:

$$h_t = f(h_{t-1}, x_t)$$

Implications:

- Cannot parallelize within training examples
- Memory constraints limit batching across examples
- Long sequences $\rightarrow$ long computation paths
- Vanishing/exploding gradients

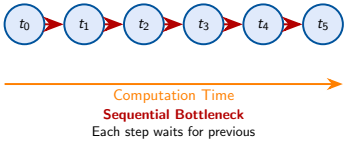
The Fundamental Problem with RNNs

Sequential Nature:

$$h_t = f(h_{t-1}, x_t)$$

Implications:

- Cannot parallelize within training examples
- Memory constraints limit batching across examples
- Long sequences $\rightarrow$ long computation paths
- Vanishing/exploding gradients



Path Length Problem

Maximum path length between any two positions: $O(n)$

This makes learning **long-range dependencies** very difficult!

The Fundamental Problem with RNNs

Sequential Nature:

$$h_t = f(h_{t-1}, x_t)$$

Implications:

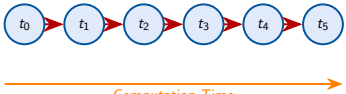
- Cannot parallelize within training examples
- Memory constraints limit batching across examples
- Long sequences $\rightarrow$ long computation paths
- Vanishing/exploding gradients

Path Length Problem

Maximum path length between any two positions: $O(n)$

This makes learning **long-range dependencies** very difficult!

Goal: Process all positions simultaneously!



Previous Solutions and Their Limitations

Factorization Tricks [Kuchaiev & Ginsburg, 2017]

- Improves computational efficiency
- Still maintains sequential nature

Previous Solutions and Their Limitations

Factorization Tricks [Kuchaiev & Ginsburg, 2017]

- Improves computational efficiency
- Still maintains sequential nature

Conditional Computation [Shazeer et al., 2017]

- MoE (Mixture of Experts)
- Improves model performance
- Doesn't solve sequential constraint

Previous Solutions and Their Limitations

Factorization Tricks [Kuchaiev & Ginsburg, 2017]

- Improves computational efficiency
- Still maintains sequential nature

Conditional Computation [Shazeer et al., 2017]

- MoE (Mixture of Experts)
- Improves model performance
- Doesn't solve sequential constraint

Convolutional Approaches

Extended Neural GPU, **ByteNet**, **ConvS2S**

- Parallel computation possible
- Path length grows with distance:
  - ConvS2S: $O(n)$ linear
  - ByteNet: $O(\log n)$ logarithmic

Previous Solutions and Their Limitations

Factorization Tricks [Kuchaiev & Ginsburg, 2017]

- Improves computational efficiency
- Still maintains sequential nature

Conditional Computation [Shazeer et al., 2017]

- MoE (Mixture of Experts)
- Improves model performance
- Doesn't solve sequential constraint

Convolutional Approaches

Extended Neural GPU, **ByteNet**, **ConvS2S**

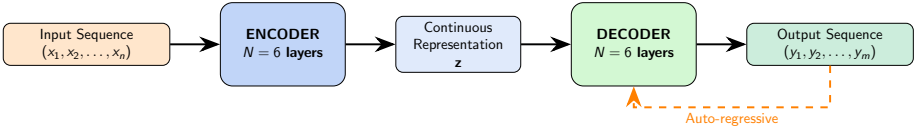
- Parallel computation possible
- Path length grows with distance:
  - ConvS2S: $O(n)$ linear
  - ByteNet: $O(\log n)$ logarithmic

The Transformer Solution

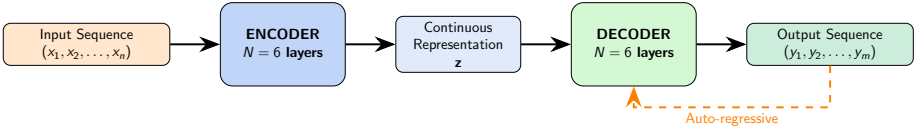
Constant path length: $O(1)$

Uses **self-attention** to connect all positions directly!

The Big Picture: Encoder-Decoder Framework



The Big Picture: Encoder-Decoder Framework



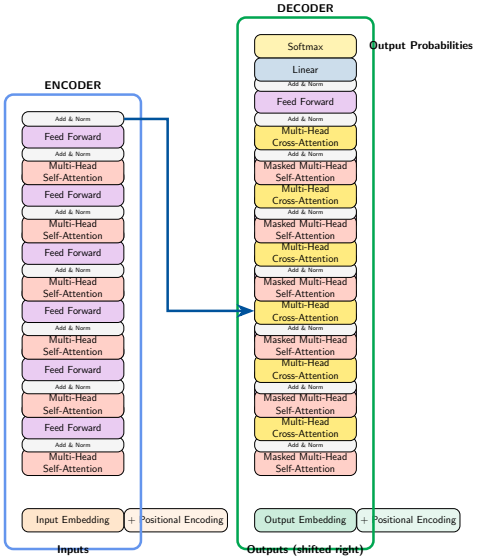
Encoder

Maps input $(x_1, \ldots, x_n)$ to continuous representations $\mathbf{z} = (z_1, \ldots, z_n)$

Decoder

Generates output $(y_1, \ldots, y_m)$ one element at a time, consuming previously generated symbols

Complete Transformer Architecture



Encoder Stack Details

Key Components:

1. **Multi-Head Self-Attention**
   - Each position attends to all positions
   - Captures global dependencies

Encoder Stack Details

Key Components:

1. **Multi-Head Self-Attention**
   - Each position attends to all positions
   - Captures global dependencies
2. **Position-wise Feed-Forward Network**
   - Applied to each position separately
   - Two linear transformations with ReLU

Encoder Stack Details

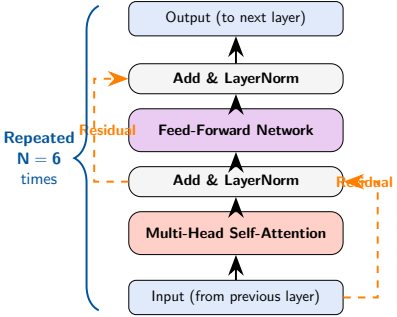
Key Components:

1. **Multi-Head Self-Attention**
   - Each position attends to all positions
   - Captures global dependencies
2. **Position-wise Feed-Forward Network**
   - Applied to each position separately
   - Two linear transformations with ReLU
3. **Residual Connections**
   - Around each sub-layer
   - Output: $\text{LayerNorm}(x + \text{Sublayer}(x))$

Encoder Stack Details

Key Components:

1. **Multi-Head Self-Attention**
   - Each position attends to all positions
   - Captures global dependencies
2. **Position-wise Feed-Forward Network**
   - Applied to each position separately
   - Two linear transformations with ReLU
3. **Residual Connections**
   - Around each sub-layer
   - Output: $\text{LayerNorm}(x + \text{Sublayer}(x))$



Dimension Requirements

All sub-layers produce outputs of dimension $d_{\text{model}} = 512$

This facilitates residual connections!

Decoder Stack Details

Three Sub-layers per Layer:

1. **Masked Multi-Head Self-Attention**
   - Prevents attending to future positions
   - Preserves auto-regressive property
   - Masking: set to $-\infty$ before softmax

Decoder Stack Details

Three Sub-layers per Layer:

1. **Masked Multi-Head Self-Attention**
   - Prevents attending to future positions
   - Preserves auto-regressive property
   - Masking: set to $-\infty$ before softmax
2. **Encoder-Decoder Attention**
   - Queries from previous decoder layer
   - Keys and Values from encoder output
   - Attends to all input positions

Decoder Stack Details

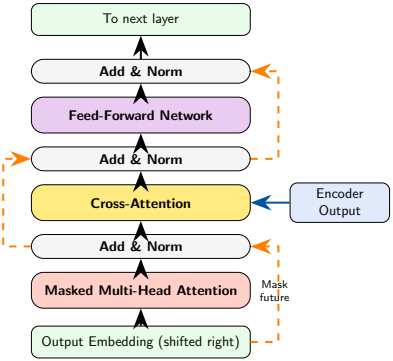
Three Sub-layers per Layer:

1. **Masked Multi-Head Self-Attention**
   - Prevents attending to future positions
   - Preserves auto-regressive property
   - Masking: set to $-\infty$ before softmax
2. **Encoder-Decoder Attention**
   - Queries from previous decoder layer
   - Keys and Values from encoder output
   - Attends to all input positions
3. **Position-wise Feed-Forward Network**
   - Same as encoder

Decoder Stack Details

Three Sub-layers per Layer:

1. **Masked Multi-Head Self-Attention**
   - Prevents attending to future positions
   - Preserves auto-regressive property
   - Masking: set to $-\infty$ before softmax
2. **Encoder-Decoder Attention**
   - Queries from previous decoder layer
   - Keys and Values from encoder output
   - Attends to all input positions
3. **Position-wise Feed-Forward Network**
   - Same as encoder



Critical: Masking

Predictions for position i can depend only on known outputs at positions less than i

What is Attention?

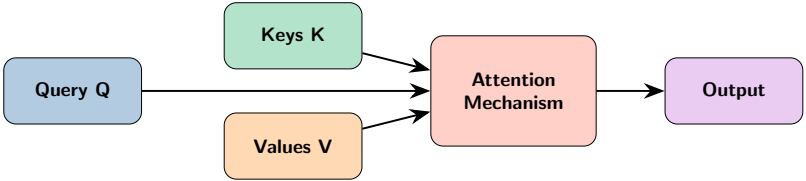
Definition

An attention function maps a **query** and a set of **key-value pairs** to an **output**, where the query, keys, values, and output are all vectors.

What is Attention?

Definition

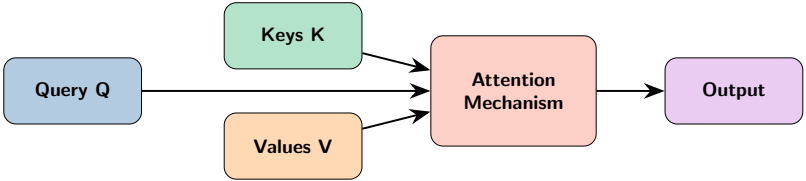
An attention function maps a **query** and a set of **key-value pairs** to an **output**, where the query, keys, values, and output are all vectors.



What is Attention?

Definition

An attention function maps a **query** and a set of **key-value pairs** to an **output**, where the query, keys, values, and output are all vectors.



Output Computation:

- Weighted sum of values
- Weights from compatibility function
- Compatibility = query-key similarity

Intuition:

- Query: What I'm looking for
- Keys: What I can match against
- Values: What I retrieve

Scaled Dot-Product Attention

The Formula:

$$\boxed{\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V}$$

Scaled Dot-Product Attention

The Formula:

$$\boxed{\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V}$$

Components:

- Q: Queries matrix, shape $(n \times d_k)$

Scaled Dot-Product Attention

The Formula:

$$\boxed{\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V}$$

Components:

- Q: Queries matrix, shape $(n \times d_k)$
- K: Keys matrix, shape $(m \times d_k)$

Scaled Dot-Product Attention

The Formula:

$$\boxed{\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V}$$

Components:

- Q: Queries matrix, shape $(n \times d_k)$
- K: Keys matrix, shape $(m \times d_k)$
- V: Values matrix, shape $(m \times d_v)$

Scaled Dot-Product Attention

The Formula:

$$\boxed{\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V}$$

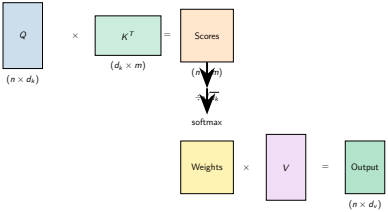
Components:

- Q: Queries matrix, shape $(n \times d_k)$
- K: Keys matrix, shape $(m \times d_k)$
- V: Values matrix, shape $(m \times d_v)$
- d_k: Dimension of keys (for scaling)

Scaled Dot-Product Attention

The Formula:

$$\boxed{\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V}$$



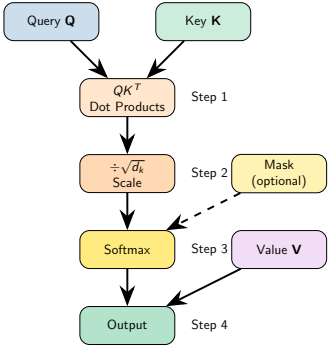
Components:

- Q: Queries matrix, shape $(n \times d_k)$
- K: Keys matrix, shape $(m \times d_k)$
- V: Values matrix, shape $(m \times d_v)$
- d_k: Dimension of keys (for scaling)

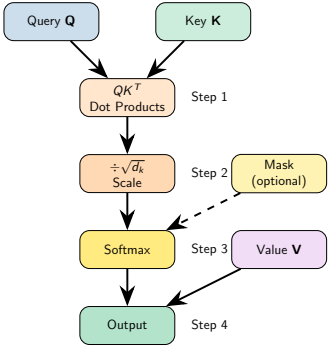
Why Scale by $\sqrt{d_k}$?

Large d_k makes dot products too large, so scaling keeps softmax gradients stable.

Scaled Dot-Product Attention: Step-by-Step



Scaled Dot-Product Attention: Step-by-Step



Quick Summary

$QK^T \to$ scale by $\sqrt{d_k} \to$ mask (decoder only) $\to$ softmax $\to$ weighted sum with V. Dot-product attention is faster and more memory-efficient than additive attention.

Multi-Head Attention: The Key Innovation

The Idea:
Instead of single attention with
d_{model}-dimensional keys/values/queries...

Multi-Head Attention: The Key Innovation

The Idea:
Instead of single attention with d_{model}-dimensional keys/values/queries...

Multi-Head Strategy

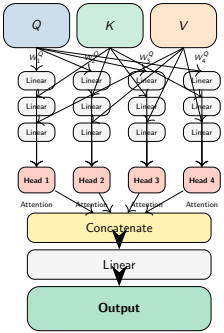
1. Linearly project h times to different subspaces
2. Apply attention in parallel
3. Concatenate results
4. Final linear projection

Multi-Head Attention: The Key Innovation

The Idea:
Instead of single attention with d_{model}-dimensional keys/values/queries...

Multi-Head Strategy

1. Linearly project h times to different subspaces
2. Apply attention in parallel
3. Concatenate results
4. Final linear projection



Formally:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W^O$$

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

Multi-Head Attention: Configuration

Paper's Configuration:

- Number of heads: $h = 8$
- Model dimension: $d_{\text{model}} = 512$
- Key dimension per head:
  $d_k = d_{\text{model}}/h = 64$
- Value dimension per head:
  $d_v = d_{\text{model}}/h = 64$

Multi-Head Attention: Configuration

Paper's Configuration:

- Number of heads: $h = 8$
- Model dimension: $d_{\text{model}} = 512$
- Key dimension per head:
  $d_k = d_{\text{model}}/h = 64$
- Value dimension per head:
  $d_v = d_{\text{model}}/h = 64$

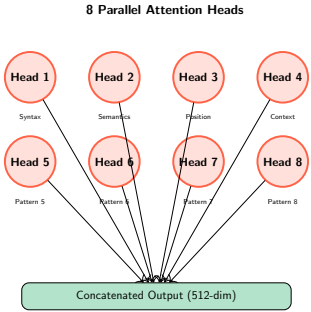
Computational Cost

Due to reduced dimension per head, total computational cost is similar to single-head attention with full dimensionality!

Multi-Head Attention: Configuration

Paper's Configuration:

- Number of heads: $h = 8$
- Model dimension: $d_{\text{model}} = 512$
- Key dimension per head: $d_k = d_{\text{model}}/h = 64$
- Value dimension per head: $d_v = d_{\text{model}}/h = 64$



Computational Cost

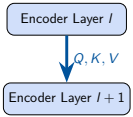
Due to reduced dimension per head, total computational cost is similar to single-head attention with full dimensionality!

Why Multiple Heads?

Allows the model to **jointly attend** to information from **different representation subspaces** at different positions.

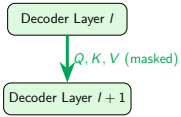
Three Types of Attention in Transformer

1. Encoder Self-Attention



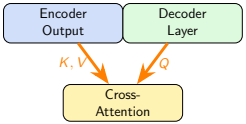
All positions attend
to all positions

2. Masked Decoder Self-Attention



Each position attends
only to earlier positions

3. Encoder-Decoder Attention



Decoder attends to
all encoder positions

Attention Visualization Example

What Attention Heads Learn:

- Different heads learn different linguistic patterns

Attention Visualization Example

What Attention Heads Learn:

- Different heads learn different linguistic patterns
- Some heads focus on **syntax** (subject-verb agreement)

Attention Visualization Example

What Attention Heads Learn:

- Different heads learn different linguistic patterns
- Some heads focus on **syntax** (subject-verb agreement)
- Some heads focus on **semantics** (word relationships)

Attention Visualization Example

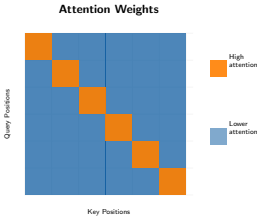
What Attention Heads Learn:

- Different heads learn different linguistic patterns
- Some heads focus on **syntax** (subject-verb agreement)
- Some heads focus on **semantics** (word relationships)
- Some heads attend to **position** (nearby words)

Attention Visualization Example

What Attention Heads Learn:

- Different heads learn different linguistic patterns
- Some heads focus on **syntax** (subject-verb agreement)
- Some heads focus on **semantics** (word relationships)
- Some heads attend to **position** (nearby words)



Interpretability Benefit

Self-attention yields more interpretable models. Individual attention heads clearly learn to perform different tasks, many exhibiting behavior related to syntactic and semantic structure.

Position-wise Feed-Forward Networks

Architecture:

$$\mathrm{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

Position-wise Feed-Forward Networks

Architecture:

$$\mathrm{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

Components:

- Two linear transformations

Position-wise Feed-Forward Networks

Architecture:

$$\mathrm{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

Components:

- Two linear transformations
- ReLU activation in between

Position-wise Feed-Forward Networks

Architecture:

$$\mathrm{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

Components:

- Two linear transformations
- ReLU activation in between
- Applied to **each position separately**

Position-wise Feed-Forward Networks

Architecture:

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

Components:

- Two linear transformations
- ReLU activation in between
- Applied to **each position separately**
- Same parameters across positions

Position-wise Feed-Forward Networks

Architecture:

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

Components:

- Two linear transformations
- ReLU activation in between
- Applied to **each position separately**
- Same parameters across positions
- Different parameters between layers

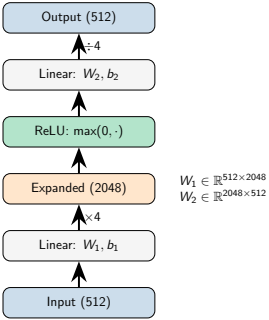
Position-wise Feed-Forward Networks

Architecture:

$$\mathrm{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

Components:

- Two linear transformations
- ReLU activation in between
- Applied to **each position separately**
- Same parameters across positions
- Different parameters between layers



Dimensions

- Input/Output: $d_{\text{model}} = 512$
- Inner layer: $d_{\text{ff}} = 2048$

Alternative View: Convolutions

Equivalence to Convolutions

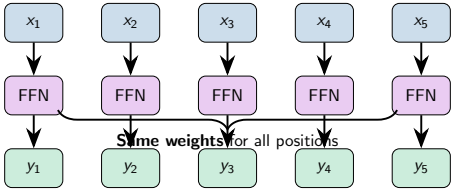
Another way of describing this is as **two convolutions with kernel size 1**.

Alternative View: Convolutions

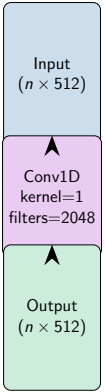
Equivalence to Convolutions

Another way of describing this is as **two convolutions with kernel size 1**.

Position-wise FFN Application



Equivalent to Conv1D



Embeddings and Softmax

Token Embeddings:

- Convert input/output tokens to vectors
- Dimension: $d_{\text{model}} = 512$
- Learned during training

Embeddings and Softmax

Token Embeddings:

- Convert input/output tokens to vectors
- Dimension: $d_{\text{model}} = 512$
- Learned during training

Weight Sharing:

- Same weight matrix between:
  - Two embedding layers
  - Pre-softmax linear transformation
- Embedding weights multiplied by $\sqrt{d_{\text{model}}}$

Embeddings and Softmax

Token Embeddings:

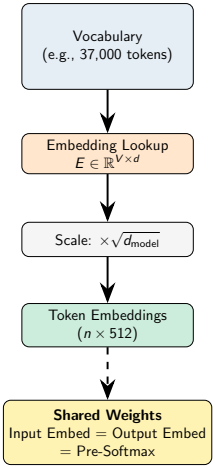
- Convert input/output tokens to vectors
- Dimension: $d_{\text{model}} = 512$
- Learned during training

Weight Sharing:

- Same weight matrix between:
  - Two embedding layers
  - Pre-softmax linear transformation
- Embedding weights multiplied by $\sqrt{d_{\text{model}}}$

Why Scale Embeddings?

To make the embeddings and positional encodings comparable in magnitude when they are summed together.



Positional Encoding: The Key to Sequence Order

The Problem

Transformer has no recurrence and no convolution. How does it know the **order** of tokens in a sequence?

Positional Encoding: The Key to Sequence Order

The Problem

Transformer has no recurrence and no convolution. How does it know the **order** of tokens in a sequence?

The Solution: Positional Encoding

Inject information about the relative or absolute position of tokens by adding **positional encodings** to input embeddings.

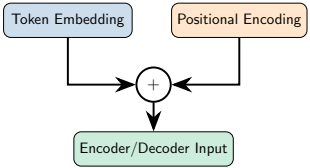
Positional Encoding: The Key to Sequence Order

The Problem

Transformer has no recurrence and no convolution. How does it know the **order** of tokens in a sequence?

The Solution: Positional Encoding

Inject information about the relative or absolute position of tokens by adding **positional encodings** to input embeddings.



Sinusoidal Positional Encoding

The Encoding Functions:

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

Sinusoidal Positional Encoding

The Encoding Functions:

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

Parameters:

- pos: position in sequence
- i: dimension index
- Each dimension = sinusoid

Sinusoidal Positional Encoding

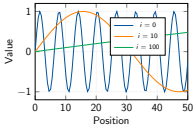
The Encoding Functions:

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

Parameters:

- pos: position in sequence
- i: dimension index
- Each dimension = sinusoid



Wavelengths

Form a **geometric progression** from 2π to $10000 \cdot 2\pi$

Sinusoidal Positional Encoding

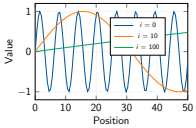
The Encoding Functions:

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

Parameters:

- pos: position in sequence
- i: dimension index
- Each dimension = sinusoid



Wavelengths

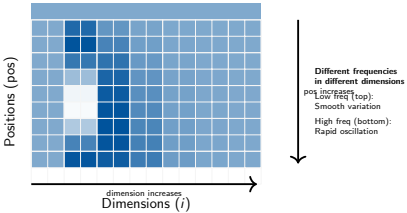
Form a **geometric progression** from 2π to $10000 \cdot 2\pi$

Key Property

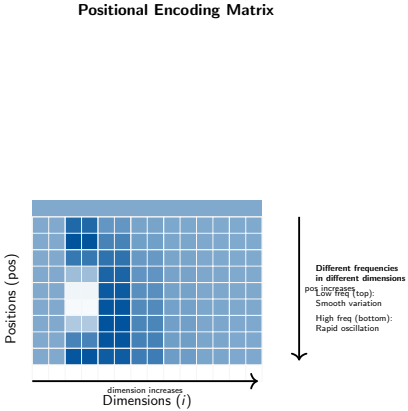
For any fixed offset k, PE_{pos+k} can be represented as a **linear function** of PE_{pos}. This allows the model to easily learn to attend by relative positions!

Positional Encoding Visualization

Positional Encoding Matrix



Positional Encoding Visualization



Extrapolation Benefit

Chose sinusoidal version because it may allow the model to **extrapolate to sequence lengths longer** than those encountered during training.

Comparison: Self-Attention vs. RNN vs. CNN

Layer Type	**Complexity per Layer**	**Sequential Ops**	**Max Path Length**
Self-Attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\log_k(n))$
Self-Attention (restricted)	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

Comparison: Self-Attention vs. RNN vs. CNN

Layer Type	Complexity per Layer	Sequential Ops	Max Path Length
Self-Attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\log_k(n))$
Self-Attention (restricted)	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

Where $n < d$

Self-attention is **faster** than recurrent layers when sequence length n is smaller than representation dimension d.

True for most state-of-the-art models!

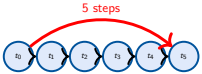
Key Advantage

Maximum path length between any two positions: $O(1)$

This is crucial for learning **long-range dependencies**!

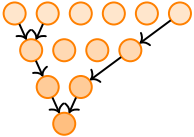
Visual Comparison: Path Lengths

RNN: $O(n)$



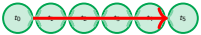
Long path for
distant positions

CNN: $O(\log n)$



Logarithmic path
(dilated convolutions)

Self-Attention: $O(1)$



Direct connection
in 1 step!

Three Desiderata for Sequence Models

1. Computational Complexity

Total computational cost per layer

Self-Attention: $O(n^2 \cdot d)$

2. Parallelization

Minimum sequential operations

Self-Attention: $O(1)$

3. Path Length

Max path for long-range dependencies

Self-Attention: $O(1)$

Three Desiderata for Sequence Models

1. Computational Complexity

Total computational cost per layer

Self-Attention: $O(n^2 \cdot d)$

2. Parallelization

Minimum sequential operations

Self-Attention: $O(1)$

3. Path Length

Max path for long-range dependencies

Self-Attention: $O(1)$

Self-Attention Wins on Two Fronts!

- **Constant path length** $\rightarrow$ easier to learn long-range dependencies
- **Constant sequential operations** $\rightarrow$ maximum parallelization
- Computational cost acceptable when $n < d$ (typical in practice)

Training Configuration

Datasets:

- **WMT 2014 English-German**
  - 4.5 million sentence pairs
  - Byte-pair encoding: 37,000 tokens
- **WMT 2014 English-French**
  - 36 million sentences
  - Word-piece: 32,000 tokens

Training Configuration

Datasets:

- **WMT 2014 English-German**
  - 4.5 million sentence pairs
  - Byte-pair encoding: 37,000 tokens
- **WMT 2014 English-French**
  - 36 million sentences
  - Word-piece: 32,000 tokens

Hardware:

- 8 NVIDIA P100 GPUs
- Base model: 0.4 sec/step
- Big model: 1.0 sec/step

Training Duration:

- Base model: 100,000 steps (12 hours)
- Big model: 300,000 steps (3.5 days)

Training Configuration

Datasets:

- **WMT 2014 English-German**
  - 4.5 million sentence pairs
  - Byte-pair encoding: 37,000 tokens
- **WMT 2014 English-French**
  - 36 million sentences
  - Word-piece: 32,000 tokens

Hardware:

- 8 NVIDIA P100 GPUs
- Base model: 0.4 sec/step
- Big model: 1.0 sec/step

Training Duration:

- Base model: 100,000 steps (12 hours)
- Big model: 300,000 steps (3.5 days)

Optimizer: Adam

- $\beta_1 = 0.9$
- $\beta_2 = 0.98$
- $\epsilon = 10^{-9}$

Training Configuration

Datasets:

- **WMT 2014 English-German**
  - 4.5 million sentence pairs
  - Byte-pair encoding: 37,000 tokens
- **WMT 2014 English-French**
  - 36 million sentences
  - Word-piece: 32,000 tokens

Hardware:

- 8 NVIDIA P100 GPUs
- Base model: 0.4 sec/step
- Big model: 1.0 sec/step

Training Duration:

- Base model: 100,000 steps (12 hours)
- Big model: 300,000 steps (3.5 days)

Optimizer: Adam

- $\beta_1 = 0.9$
- $\beta_2 = 0.98$
- $\epsilon = 10^{-9}$

Learning Rate Schedule:

$$lrate = d_{\text{model}}^{-0.5} \cdot \min(\ldots)$$

- Warmup: 4,000 steps
- Linear increase then inverse sqrt decay

Regularization Techniques

1. Residual Dropout

Applied to:

- Output of each sub-layer
- Sums of embeddings and positional encodings

Rate: $P_{drop} = 0.1$ (base model)

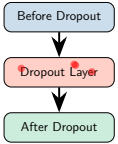
Regularization Techniques

1. Residual Dropout

Applied to:

- Output of each sub-layer
- Sums of embeddings and positional encodings

Rate: $P_{drop} = 0.1$ (base model)



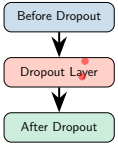
Regularization Techniques

1. Residual Dropout

Applied to:

- Output of each sub-layer
- Sums of embeddings and positional encodings

Rate: $P_{drop} = 0.1$ (base model)



2. Label Smoothing

- Value: $\epsilon_{ls} = 0.1$
- Hurts perplexity
- Improves accuracy and BLEU

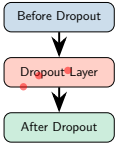
Regularization Techniques

1. Residual Dropout

Applied to:

- Output of each sub-layer
- Sums of embeddings and positional encodings

Rate: $P_{drop} = 0.1$ (base model)



2. Label Smoothing

- Value: $\epsilon_{ls} = 0.1$
- Hurts perplexity
- Improves accuracy and BLEU

Why Label Smoothing?

Makes the model learn to be more uncertain, which:

- Improves generalization
- Prevents overconfident predictions
- Better calibration

Results: Machine Translation Performance

Model	EN-DE BLEU	EN-FR BLEU	Training Cost
ByteNet [15]	23.75	–	–
Deep-Att + PosUnk [32]	–	39.2	1.0×10^{20}
GNMT + RL [31]	24.6	39.92	2.3×10^{19}
ConvS2S [8]	25.16	40.46	9.6×10^{18}
MoE [26]	26.03	40.56	2.0×10^{19}
GNMT + RL Ensemble	26.30	41.16	1.8×10^{20}
ConvS2S Ensemble	26.36	41.29	7.7×10^{19}
Transformer (base)	**27.3**	38.1	$\mathbf{3.3 \times 10^{18}}$
Transformer (big)	**28.4**	**41.0**	2.3×10^{19}

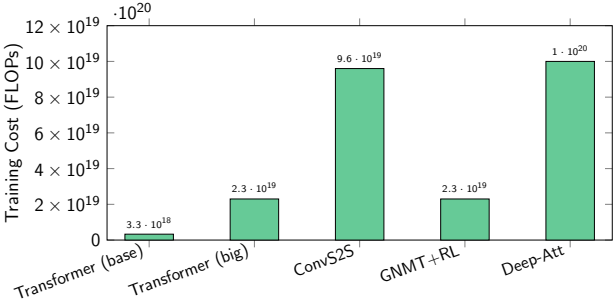
Results: Machine Translation Performance

Model	EN-DE BLEU	EN-FR BLEU	Training Cost
ByteNet [15]	23.75	–	–
Deep-Att + PosUnk [32]	–	39.2	1.0×10^{20}
GNMT + RL [31]	24.6	39.92	2.3×10^{19}
ConvS2S [8]	25.16	40.46	9.6×10^{18}
MoE [26]	26.03	40.56	2.0×10^{19}
GNMT + RL Ensemble	26.30	41.16	1.8×10^{20}
ConvS2S Ensemble	26.36	41.29	7.7×10^{19}
Transformer (base)	**27.3**	38.1	$\mathbf{3.3 \times 10^{18}}$
Transformer (big)	**28.4**	**41.0**	2.3×10^{19}

Takeaway

Transformer-big reaches **28.4** (EN-DE) and **41.0** (EN-FR) BLEU with much lower training cost than prior systems.

Training Cost Comparison



Key Insight

Transformer (base) achieves **better results** than all previous models at a **fraction of the training cost**!

Model Variations: Ablation Study

	Variation	PPL (dev)	BLEU (dev)	Params
	Base model	**4.92**	**25.8**	65M
(A)	$h = 1$	5.29	24.9	–
	$h = 4$	5.00	25.5	–
	$h = 16$	4.91	25.8	–
	$h = 32$	5.01	25.4	–
(B)	$d_k = d_v = 16$	5.16	25.1	58M
	$d_k = d_v = 32$	5.01	25.4	60M
(C)	$d_{model} = 1024$	4.66	26.0	168M
	$d_{ff} = 4096$	4.75	26.2	90M
(D)	Dropout $= 0.0$	5.77	24.6	–
	Dropout $= 0.2$	4.95	25.5	–
(E)	Learned positional embedding	4.92	25.7	–

Model Variations: Ablation Study

	Variation	PPL (dev)	BLEU (dev)	Params
	Base model	**4.92**	**25.8**	65M
(A)	$h = 1$	5.29	24.9	–
	$h = 4$	5.00	25.5	–
	$h = 16$	4.91	25.8	–
	$h = 32$	5.01	25.4	–
(B)	$d_k = d_v = 16$	5.16	25.1	58M
	$d_k = d_v = 32$	5.01	25.4	60M
(C)	$d_{model} = 1024$	4.66	26.0	168M
	$d_{ff} = 4096$	4.75	26.2	90M
(D)	Dropout $= 0.0$	5.77	24.6	–
	Dropout $= 0.2$	4.95	25.5	–
(E)	Learned positional embedding	4.92	25.7	–

Takeaway

Best trade-off uses multi-head attention (not single-head), adequate key/value size, and dropout around 0.1.

Key Findings from Ablation

Attention Heads

- Single head: worse quality
- Too many heads: diminishing returns
- $h = 8$ **is optimal**

Key Findings from Ablation

Attention Heads

- Single head: worse quality
- Too many heads: diminishing returns
- $h = 8$ **is optimal**

Model Size

- Bigger models are better
- $d_{model} = 1024$: 26.0 BLEU
- $d_{ff} = 4096$: 26.2 BLEU

Key Findings from Ablation

Attention Heads

- Single head: worse quality
- Too many heads: diminishing returns
- $h = 8$ **is optimal**

Dropout is Critical!

- No dropout: 24.6 BLEU
- With dropout: 25.8 BLEU
- Essential for avoiding overfitting

Model Size

- Bigger models are better
- $d_{model} = 1024$: 26.0 BLEU
- $d_{ff} = 4096$: 26.2 BLEU

Key Findings from Ablation

Attention Heads

- Single head: worse quality
- Too many heads: diminishing returns
- $h = 8$ **is optimal**

Dropout is Critical!

- No dropout: 24.6 BLEU
- With dropout: 25.8 BLEU
- Essential for avoiding overfitting

Model Size

- Bigger models are better
- $d_{model} = 1024$: 26.0 BLEU
- $d_{ff} = 4096$: 26.2 BLEU

Positional Encoding

Learned vs. Sinusoidal: **Nearly identical results!**

Sinusoidal may allow longer sequence extrapolation.

Key Contributions

1. Novel Architecture

- First sequence transduction model based **entirely on attention**
- No recurrence, no convolution
- Simple, elegant design

Key Contributions

1. Novel Architecture

- First sequence transduction model based **entirely on attention**
- No recurrence, no convolution
- Simple, elegant design

2. Computational Efficiency

- Highly parallelizable
- Significantly faster training
- Lower computational cost

Key Contributions

1. Novel Architecture

- First sequence transduction model based **entirely on attention**
- No recurrence, no convolution
- Simple, elegant design

2. Computational Efficiency

- Highly parallelizable
- Significantly faster training
- Lower computational cost

3. State-of-the-Art Results

- EN-DE: 28.4 BLEU (new SOTA)
- EN-FR: 41.0 BLEU (new SOTA)
- Trained in just 3.5 days

Key Contributions

1. Novel Architecture

- First sequence transduction model based **entirely on attention**
- No recurrence, no convolution
- Simple, elegant design

2. Computational Efficiency

- Highly parallelizable
- Significantly faster training
- Lower computational cost

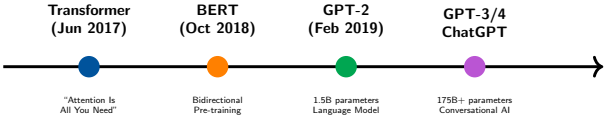
3. State-of-the-Art Results

- EN-DE: 28.4 BLEU (new SOTA)
- EN-FR: 41.0 BLEU (new SOTA)
- Trained in just 3.5 days

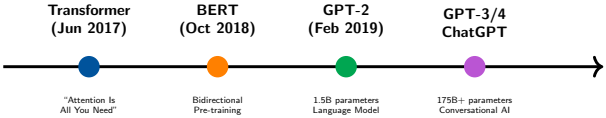
4. Interpretability

- Attention heads learn different patterns
- Clear syntactic/semantic behavior
- More interpretable than RNNs

The Transformer Revolution



The Transformer Revolution



The Foundation of Modern NLP

The Transformer architecture has become the foundation for virtually all state-of-the-art NLP models:

- **BERT, RoBERTa, ALBERT** - Understanding tasks
- **GPT series** - Generation tasks
- **T5, BART** - Text-to-text frameworks
- **Vision Transformer (ViT)** - Computer vision
- **GPT-4, Claude, Gemini** - Large Language Models

Future Directions (From the Paper)

Extending Applications

Apply Transformer to other tasks:

- Image processing
- Audio processing
- Video understanding

Future Directions (From the Paper)

Extending Applications

Apply Transformer to other tasks:

- Image processing
- Audio processing
- Video understanding

Multi-Modal Extensions

Problems involving input/output modalities other than text:

- Image captioning
- Visual question answering
- Speech-to-text

Future Directions (From the Paper)

Extending Applications

Apply Transformer to other tasks:

- Image processing
- Audio processing
- Video understanding

Efficient Attention

Local, restricted attention mechanisms:

- Handle large inputs/outputs
- Reduce $O(n^2)$ complexity
- Sparse attention patterns

Multi-Modal Extensions

Problems involving input/output modalities other than text:

- Image captioning
- Visual question answering
- Speech-to-text

Future Directions (From the Paper)

Extending Applications

Apply Transformer to other tasks:

- Image processing
- Audio processing
- Video understanding

Efficient Attention

Local, restricted attention mechanisms:

- Handle large inputs/outputs
- Reduce $O(n^2)$ complexity
- Sparse attention patterns

Multi-Modal Extensions

Problems involving input/output modalities other than text:

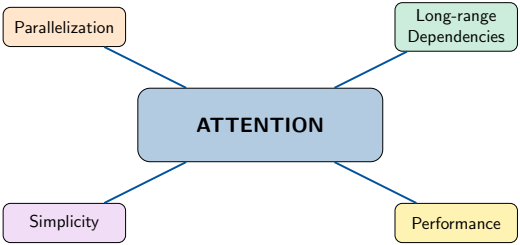
- Image captioning
- Visual question answering
- Speech-to-text

Making Generation Less Sequential

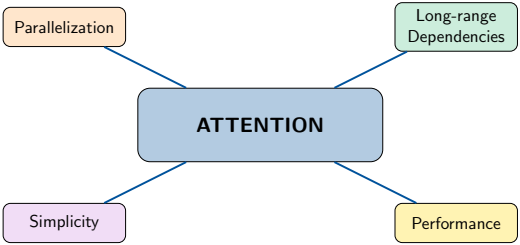
Research goal: parallel generation

- Non-autoregressive models
- Iterative refinement
- Speculative decoding

Summary: Why “Attention Is All You Need”



Summary: Why "Attention Is All You Need"



The Core Message

Attention mechanisms alone – without recurrence or convolution – are sufficient for high-quality sequence transduction.

Key Takeaways

1. **Self-Attention** enables direct connections between any positions
   - $O(1)$ path length for long-range dependencies
   - $O(1)$ sequential operations for parallelization

Key Takeaways

1. **Self-Attention** enables direct connections between any positions
   - $O(1)$ path length for long-range dependencies
   - $O(1)$ sequential operations for parallelization
2. **Multi-Head Attention** allows attending to different representation subspaces
   - 8 heads with 64 dimensions each
   - Different heads learn different patterns

Key Takeaways

1. **Self-Attention** enables direct connections between any positions
   - $O(1)$ path length for long-range dependencies
   - $O(1)$ sequential operations for parallelization
2. **Multi-Head Attention** allows attending to different representation subspaces
   - 8 heads with 64 dimensions each
   - Different heads learn different patterns
3. **Positional Encoding** injects sequence order information
   - Sinusoidal functions of different frequencies
   - Enables extrapolation to longer sequences

Key Takeaways

1. **Self-Attention** enables direct connections between any positions
   - $O(1)$ path length for long-range dependencies
   - $O(1)$ sequential operations for parallelization
2. **Multi-Head Attention** allows attending to different representation subspaces
   - 8 heads with 64 dimensions each
   - Different heads learn different patterns
3. **Positional Encoding** injects sequence order information
   - Sinusoidal functions of different frequencies
   - Enables extrapolation to longer sequences
4. **Encoder-Decoder Attention** bridges source and target
   - Decoder attends to all encoder positions
   - Key for translation quality

Key Takeaways

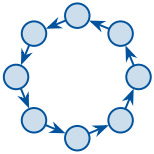
1. **Self-Attention** enables direct connections between any positions
   - $O(1)$ path length for long-range dependencies
   - $O(1)$ sequential operations for parallelization
2. **Multi-Head Attention** allows attending to different representation subspaces
   - 8 heads with 64 dimensions each
   - Different heads learn different patterns
3. **Positional Encoding** injects sequence order information
   - Sinusoidal functions of different frequencies
   - Enables extrapolation to longer sequences
4. **Encoder-Decoder Attention** bridges source and target
   - Decoder attends to all encoder positions
   - Key for translation quality
5. **Residual connections + LayerNorm** enable training deep networks
   - 6 layers in encoder and decoder
   - Stable gradient flow

References

- Vaswani, A., et al. (2017). Attention Is All You Need. *NIPS 2017*.
- Bahdanau, D., et al. (2014). Neural Machine Translation by Jointly Learning to Align and Translate. *arXiv:1409.0473*.
- Gehring, J., et al. (2017). Convolutional Sequence to Sequence Learning. *arXiv:1705.03122*.
- Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8).
- Wu, Y., et al. (2016). Google's Neural Machine Translation System. *arXiv:1609.08144*.
- Devlin, J., et al. (2018). BERT: Pre-training of Deep Bidirectional Transformers. *arXiv:1810.04805*.
- Radford, A., et al. (2018). Improving Language Understanding by Generative Pre-Training. *OpenAI*.

Thank You!

Questions?



Code: `https://github.com/tensorflow/tensor2tensor`